

CSC-3TC33

Algorithmique et fondements de l'informatique

Graphes - partie b

Bertrand Meyer

Automne 2024

Poids d'un chemin

Poids d'un chemin

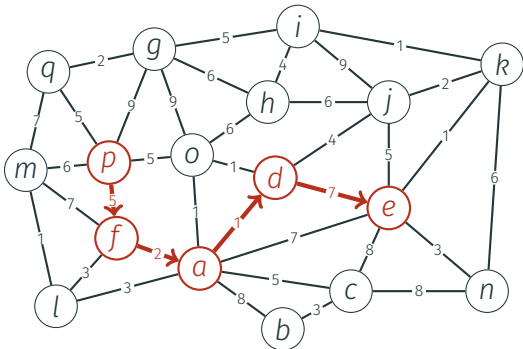
Définition

Soit $G = (V, E, \rho)$ un graphe pondéré, avec $\rho : E \rightarrow \mathbb{R}$. Le **poids d'un chemin** $C = x_0 \cdots x_k$ est la quantité

$$\rho(C) = \sum_{i=1}^{k-1} \rho(x_i x_{i+1}).$$

Example

Le chemin $C = \text{pfade}$ a pour poids $\rho(C) = 5 + 2 + 1 + 7 = 15$.



Définition

Soit $G = (V, E, \rho)$ un graphe pondéré sans circuits de poids strictement négatif. La **distance** entre $u \in V$ et $v \in V$ est

$$d(u, v) = \begin{cases} \min\{\rho(C); C \text{ chemin de } u \text{ vers } v\} & \text{si } u \rightsquigarrow v \\ +\infty & \text{sinon} \end{cases} .$$



Définition

Soit $G = (V, E, \rho)$ un graphe pondéré sans circuits de poids strictement négatif. La **distance** entre $u \in V$ et $v \in V$ est

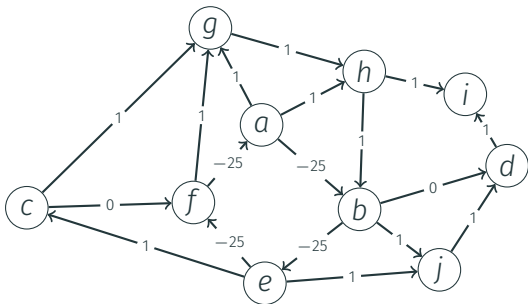
$$d(u, v) = \begin{cases} \min\{\rho(C); C \text{ chemin de } u \text{ vers } v\} & \text{si } u \rightsquigarrow v \\ +\infty & \text{sinon} \end{cases} .$$

Un **plus court chemin** est un chemin de u vers v de poids $d(u, v)$.

Circuit de poids négatif

Exemple

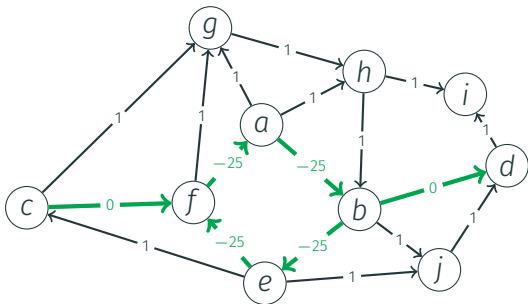
Entre c et d,



Circuit de poids négatif

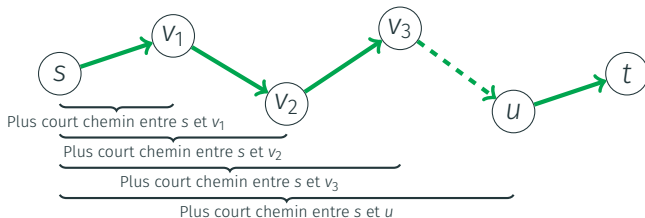
Exemple

Entre c et d , il n'y a pas de plus court chemin.



Le long d'un plus court chemin

Un plus court chemin $sv_1v_2 \cdots t$ entre s et t donne aussi un plus court chemin entre s et v_1, v_2 , etc.



Observation

$$\forall t \in V \setminus \{s\}, \quad d(s, t) = d(s, u) + \rho(ut)$$

Conséquence

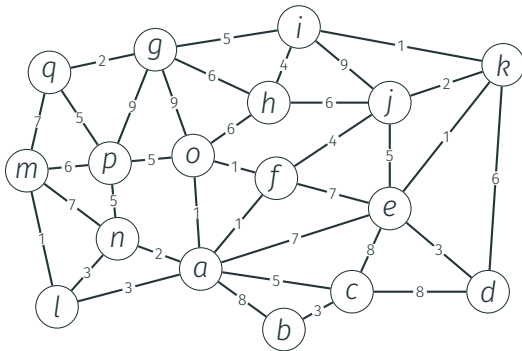
$$\forall t \in V \setminus \{s\}, \quad d(s, t) = \min_{ut.q. uv \in E} (d(s, u) + \rho(ut)). \quad (*)$$

Arborescence de plus courts chemins

Il existe une **arborescence** $\mathcal{A}$ telle que, pour tout $v \in V$, le chemin de s vers v dans $\mathcal{A}$ est de poids $d(s, v)$.

Exemple

L'arborescence des plus courts chemins depuis o est



Représentation

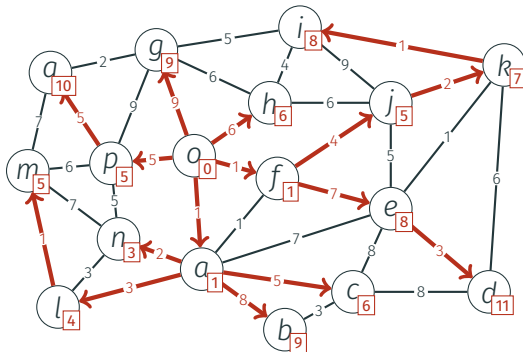
Dictionnaire : sommet $\mapsto$ père & distance.

Arborescence de plus courts chemins

Il existe une **arborescence** $\mathcal{A}$ telle que, pour tout $v \in V$, le chemin de s vers v dans $\mathcal{A}$ est de poids $d(s, v)$.

Example

L'arborescence des plus courts chemins depuis o est



Représentation

Dictionnaire : sommet $\mapsto$ père & distance.

Le cas d'un graphe acyclique orienté

Années 50



Richard Ernest Bellman
(1920†1984)

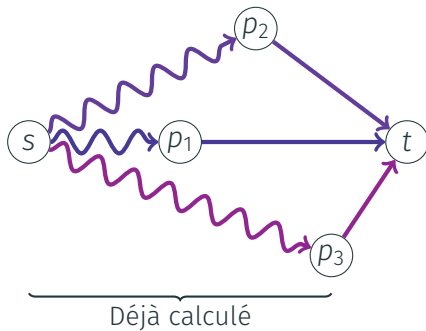
Principe

- Trier les sommets accessibles dans l'ordre topologique : $v_1 = s, v_2, \dots, v_n$ est possible !
- Calculer successivement les plus courts chemins de s à v_i grâce à $(*)$

$$\text{dist}(s, v_i) = \min_{v_j \text{ préd de } v_i} \text{dist}(s, v_j) + \rho(v_j v_i).$$

Algorithme de programmation dynamique.

Illustration de la preuve



Algorithme de Bellman : pseudo-code

Algorithme 1 : Algorithme de Bellman

Entrées : Graphe orienté acyclique pondéré $G = (V, E, \rho)$,
sommet s

Résultat : Arborescence de plus courts chemins issus de s

- 1 Numéroter les sommets accessibles depuis s dans l'ordre topologique : $s = v_0, v_1, \dots, v_k$
 - 2 $d(s) \leftarrow 0$
 - 3 **pour** $i \in \llbracket 1, k \rrbracket$ **faire**
 - 4 $d(v_i) \leftarrow \min_{v_j \text{ prédecesseur de } v_i} d(v_j) + \rho(v_j v_i)$
 - 5 $\text{père}(v_i) \leftarrow \operatorname{argmin}_{v_j \text{ prédecesseur de } v_i} d(v_j) + \rho(v_j v_i)$
-

Algorithme de Bellman : pseudo-code

Algorithme 2 : Algorithme de Bellman

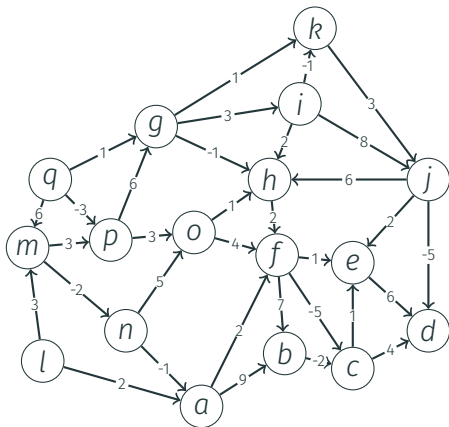
Entrées : Graphe orienté acyclique pondéré $G = (V, E, \rho)$,
sommet s

Résultat : Arborescence de plus courts chemins issus de s

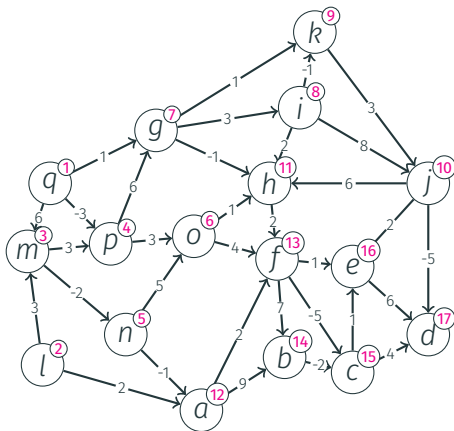
- 1 Numéroter les sommets accessibles depuis s dans l'ordre topologique : $s = v_0, v_1, \dots, v_k$
 - 2 $d(s) \leftarrow 0$
 - 3 **pour** $i \in \llbracket 1, k \rrbracket$ **faire**
 - 4 $d(v_i) \leftarrow \min_{v_j \text{ prédecesseur de } v_i} d(v_j) + \rho(v_j v_i)$
 - 5 $\text{père}(v_i) \leftarrow \operatorname{argmin}_{v_j \text{ prédecesseur de } v_i} d(v_j) + \rho(v_j v_i)$
-

Complexité : $O(n + m)$

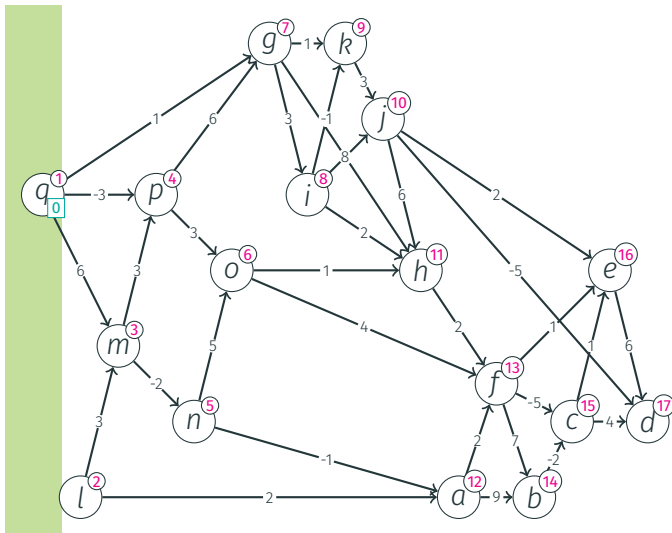
Algorithme de Bellman : illustration



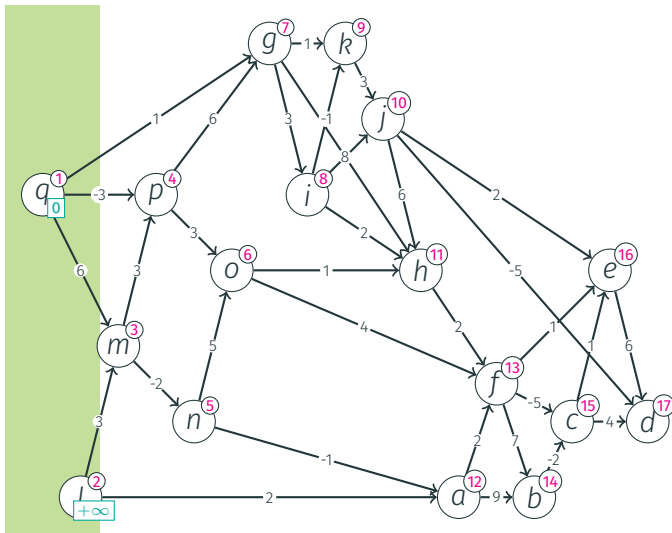
Algorithme de Bellman : illustration



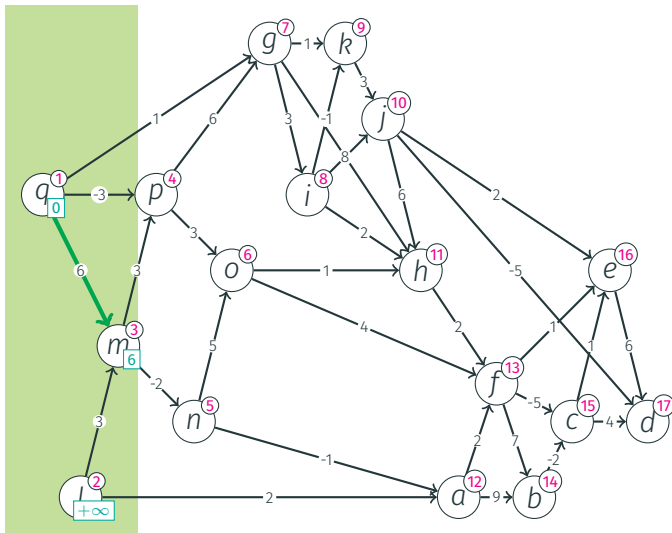
Algorithme de Bellman : illustration



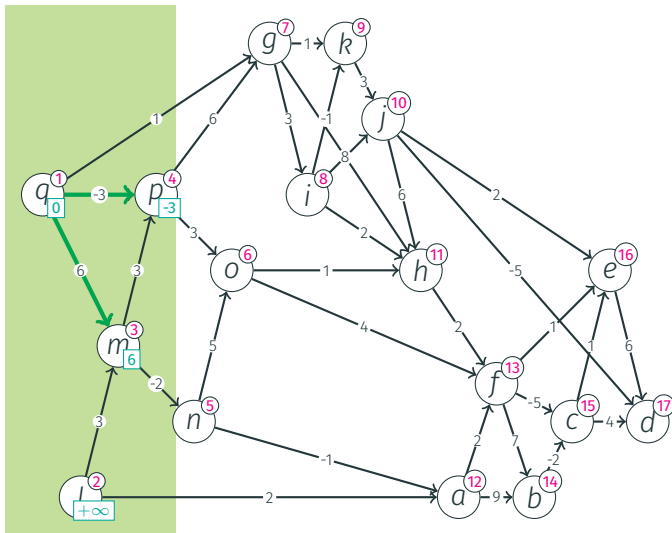
Algorithme de Bellman : illustration



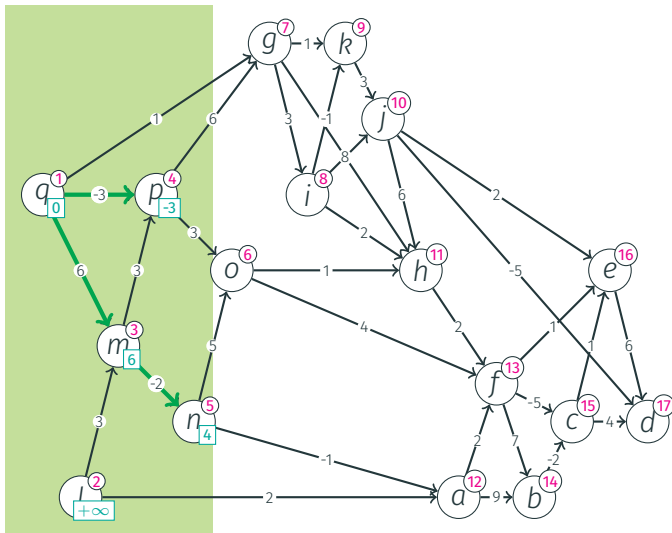
Algorithme de Bellman : illustration



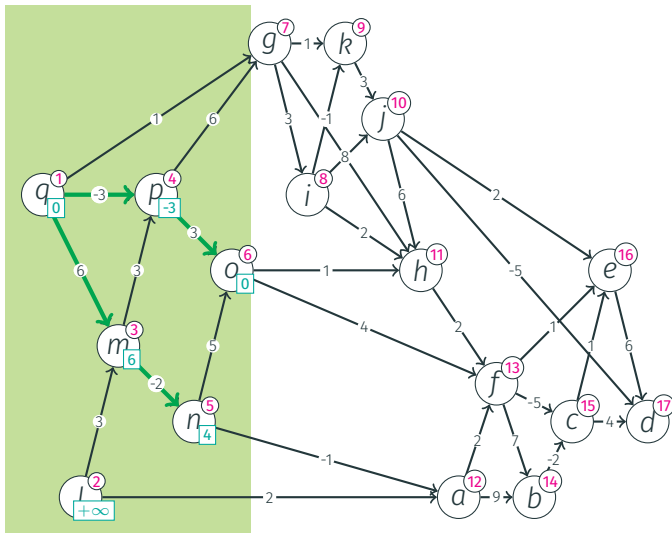
Algorithme de Bellman : illustration



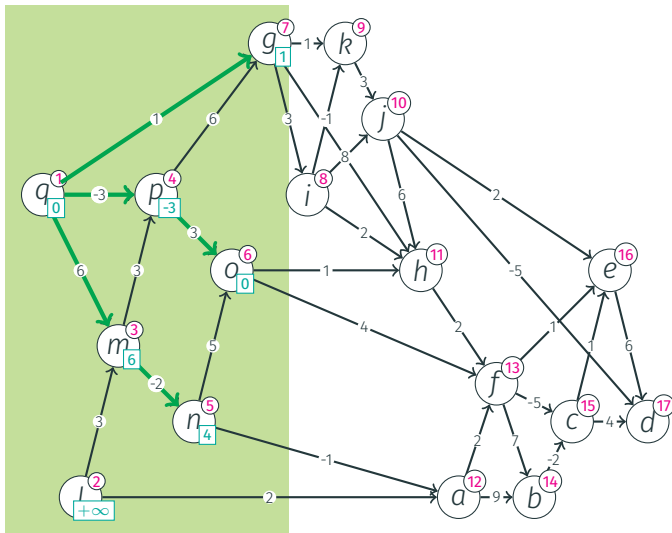
Algorithme de Bellman : illustration



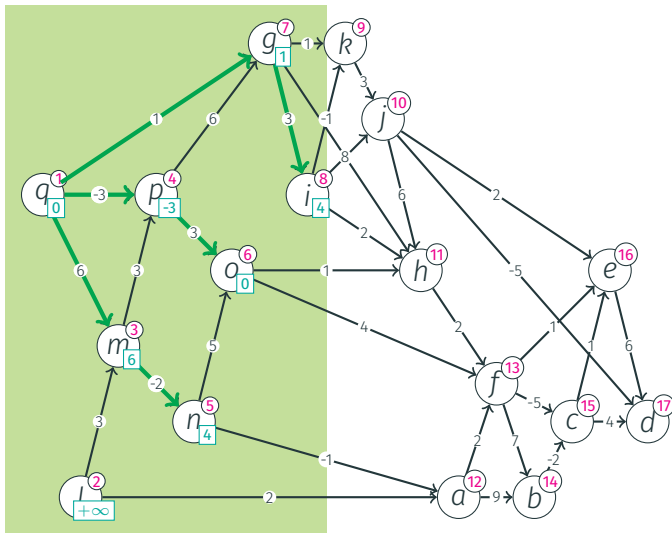
Algorithme de Bellman : illustration



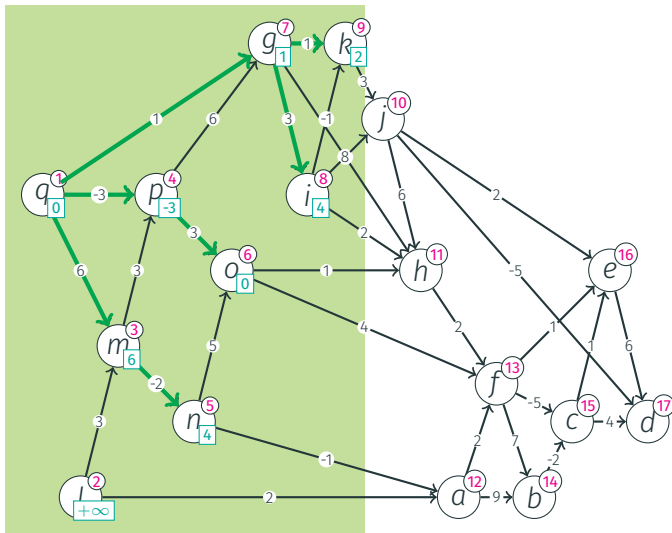
Algorithme de Bellman : illustration



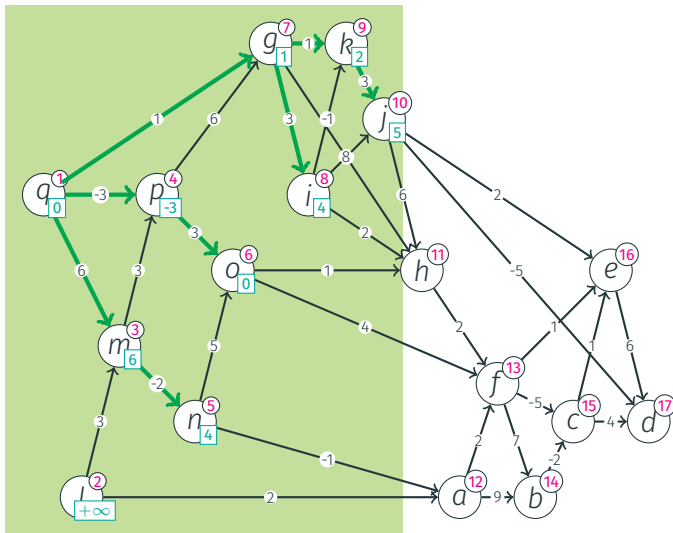
Algorithme de Bellman : illustration



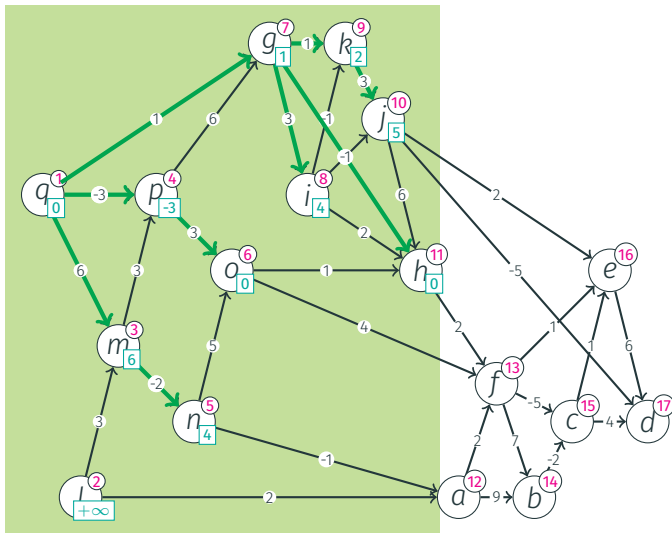
Algorithme de Bellman : illustration



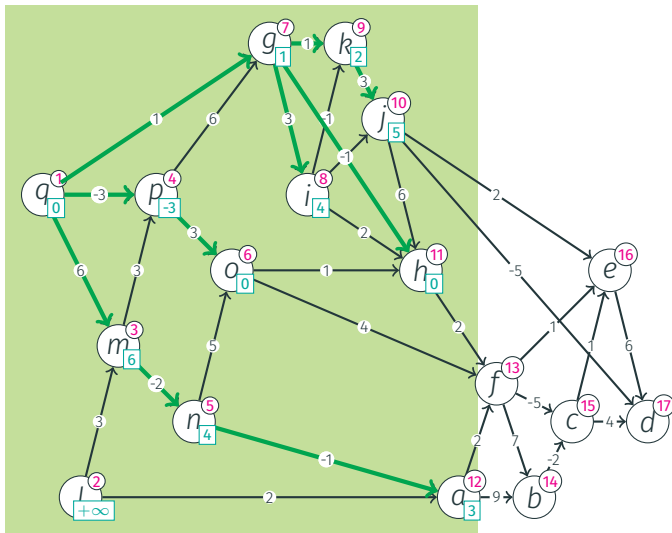
Algorithme de Bellman : illustration



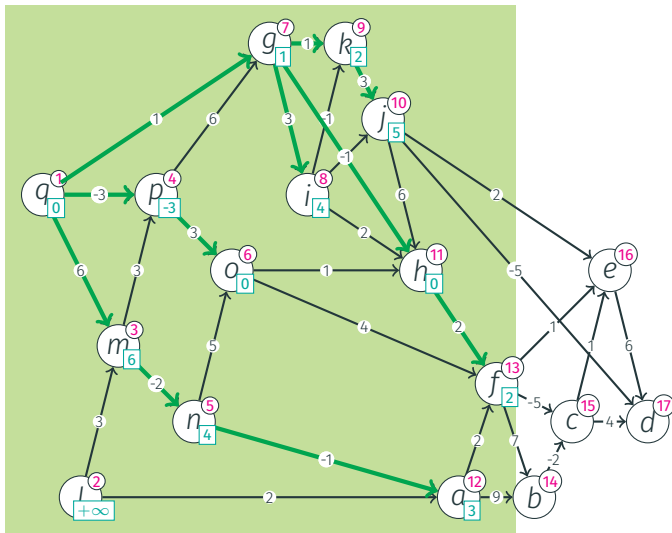
Algorithme de Bellman : illustration



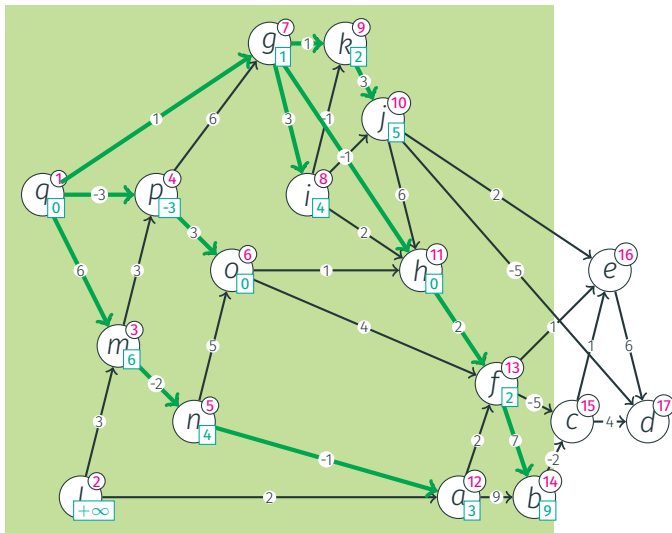
Algorithme de Bellman : illustration



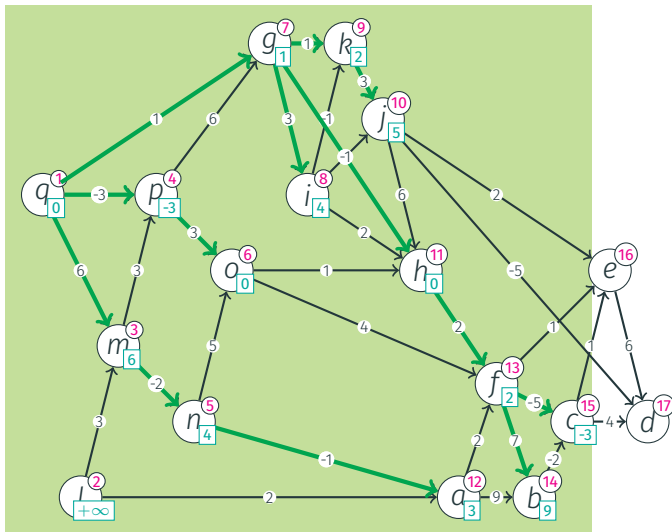
Algorithme de Bellman : illustration



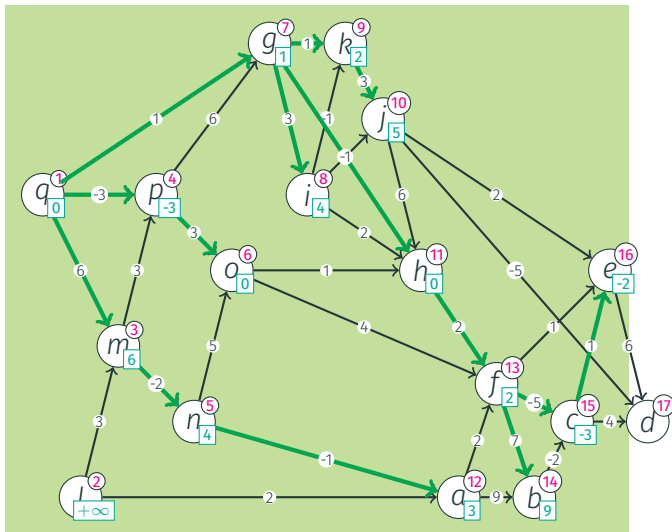
Algorithme de Bellman : illustration



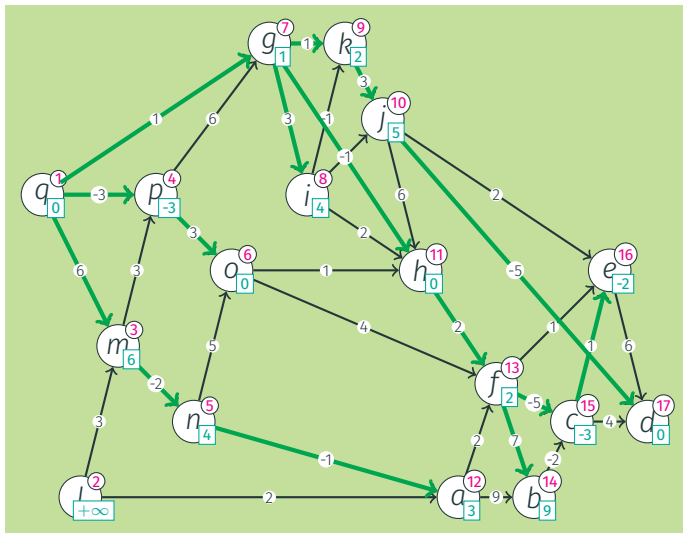
Algorithme de Bellman : illustration



Algorithme de Bellman : illustration



Algorithme de Bellman : illustration



Le cas de pondérations positives

Algorithme de Dijkstra : idée

Hypothèse : $\rho(uv) \geq 0$ pour tout arc uv .

Algorithme de Dijkstra : idée

Hypothèse : $\rho(uv) \geq 0$ pour tout arc uv .

Idée

- Si l'on savait numéroter les sommets $v_1, v_2, \dots, v_n$ par ordre de distance croissante

$$d(s, v_1) \leq d(s, v_2) \leq \dots \leq d(s, v_n)$$

- On pourrait exploiter (*) et calculer

$$\text{dist}(s, v_i) = \min_{v_j \text{ préd de } v_i} \text{dist}(s, v_j) + \rho(v_j v_i).$$

Algorithme de Dijkstra : idée

1959

Hypothèse : $\rho(uv) \geq 0$ pour tout arc uv .

Idée

- Si l'on savait numéroté les sommets $v_1, v_2, \dots, v_n$ par ordre de distance croissante

$$d(s, v_1) \leq d(s, v_2) \leq \dots \leq d(s, v_n)$$

- On pourrait exploiter (*) et calculer

$$\text{dist}(s, v_i) = \min_{v_j \text{ préd de } v_i} \text{dist}(s, v_j) + \rho(v_j v_i).$$

Intuition

- À l'étape i : Il suffit d'avoir découvert les sommets $v_1, \dots, v_{i-1}$ pour calculer $\text{dist}(s, v_i)$
- Initiation : Forcément $v_1 = s$.



Edsger Wybe Dijkstra
(1930+2002)

Algorithme de Dijkstra : principe

1959



Edsger Wybe Dijkstra
(1930+2002)

Principe

On fait évoluer une **boucle** obéissant aux **invariants**

($\boxtimes_1$) Une **arborescence partielle** père : $V \rightarrow V$ de chemins de racine s et **distance** $\pi(v)$ entre s et v est construite.

($\boxtimes_2$) L'équation $(*)$ est partiellement vraie, càd :

$$\pi(v) = \min_{p \text{ préd de } v \in \text{Dom}(\text{père})} \pi(p) + \rho(pv)$$

($\boxtimes_3$) On a $\pi(v) = d(s, v)$ pour $v \in \{v_1, \dots, v_i\}$ à l'étape i .

Algorithme 3 : Algorithme de Dijkstra

Entrées : Graphe orienté pondéré $G = (V, E, \rho)$, sommet s

- 1 **pour** $v \in V \setminus \{s\}$ **faire**
 - 2 $\pi(v) \leftarrow +\infty$
 - 3 $\pi(s) \leftarrow 0$
-

Algorithme de Dijkstra : pseudo-code

Algorithme 4 : Algorithme de Dijkstra

Entrées : Graphe orienté pondéré $G = (V, E, \rho)$, sommet s

Résultat : Distance du sommet s à tout sommet accessible, arborescence des plus courts chemins

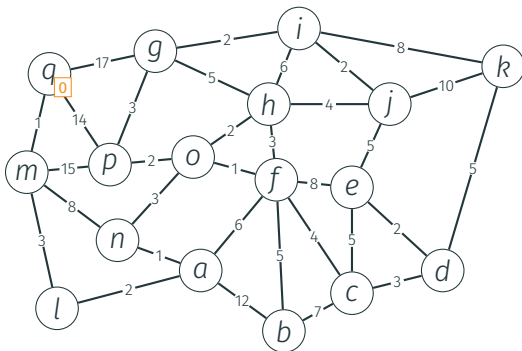
```
1 [Suite]
2  $F \leftarrow \{s\}$  : file à priorité, où la priorité du sommet  $v$  est  $\pi(v)$ 
3 tant que  $F \neq \emptyset$  faire
4      $v \leftarrow$  Extraire de  $F$  le sommet de plus petite priorité  $\pi(v)$ 
5     Marquer  $v$ 
6     pour  $w$  successeur de  $v$  non marqué faire
7         si  $\pi(v) + \rho(vw) < \pi(w)$  alors
8             si  $w \in F$  alors
9                 Diminuer la priorité  $\pi(w) \leftarrow \pi(v) + \rho(vw)$  dans  $F$ 
10            sinon
11                Insérer  $w$  à  $F$  avec la priorité  $\pi(w) \leftarrow \pi(v) + \rho(vw)$ 
12            père( $w$ )  $\leftarrow v$ 
13 retourner  $\pi$  et père
```

Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[q]$

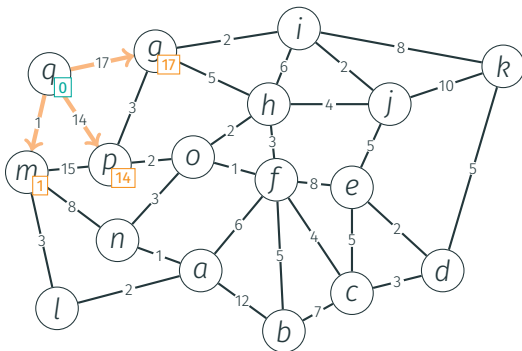


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[m, p, g]$. On étudie les voisins de q .

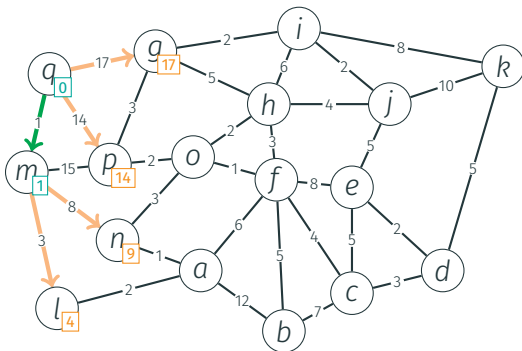


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[l, n, p, g]$. On étudie les voisins de m .

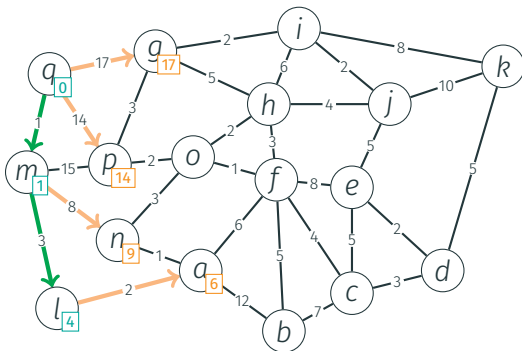


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[a,n,p,g]$. On étudie les voisins de l .

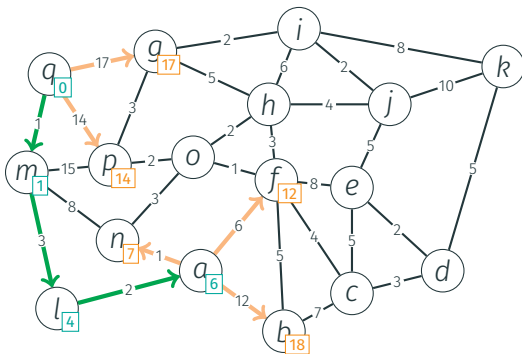


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[n, f, p, g, b]$. On étudie les voisins de a .



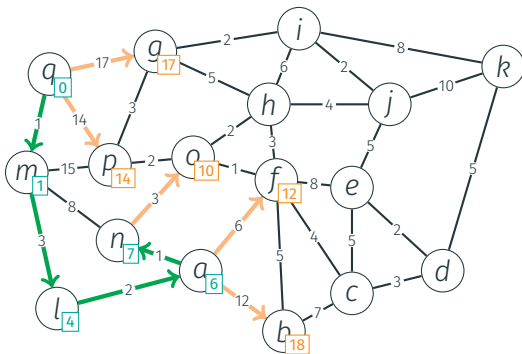
Algorithme de Dijkstra : illustration



Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[o, f, p, g, b]$. On étudie les voisins de n .

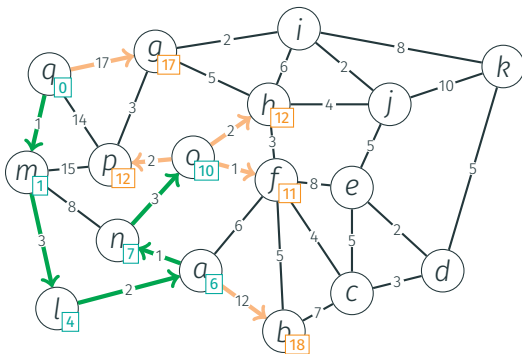


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[f,p,h,g,b]$. On étudie les voisins de o .

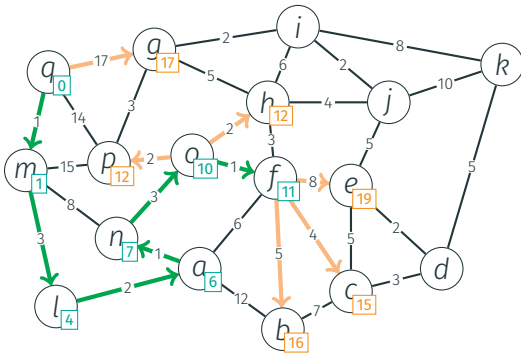


Algorithme de Dijkstra : illustration

Example

Plus courts chemins issus du sommet q .

État de la file à priorité : [p,h,c,g,b,e]. On étudie les voisins de f .

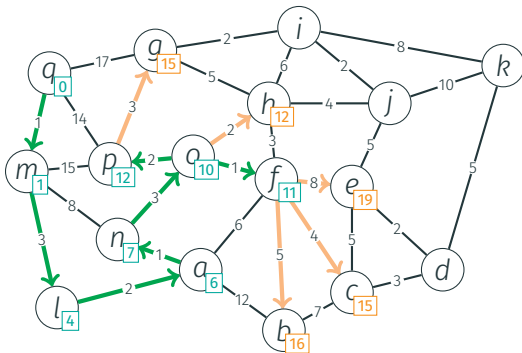


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[h, c, g, b, e]$. On étudie les voisins de p .

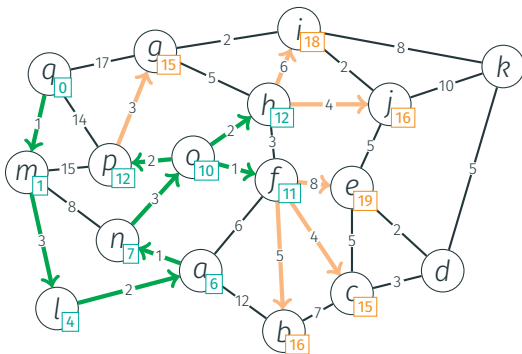


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[g,c,j,b,i,e]$. On étudie les voisins de h .

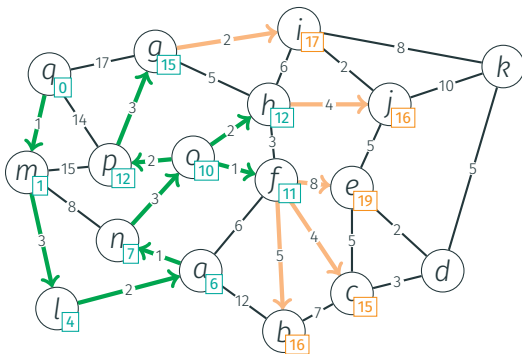


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[c,j,b,i,e]$. On étudie les voisins de g .

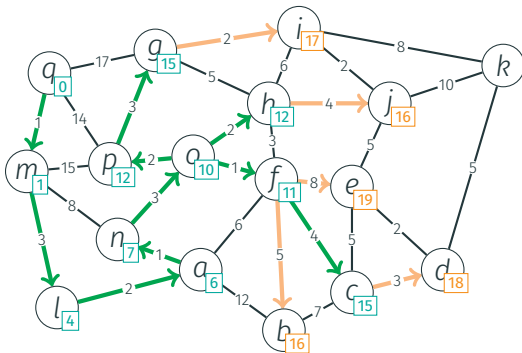


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[j,b,i,d,e]$. On étudie les voisins de c .

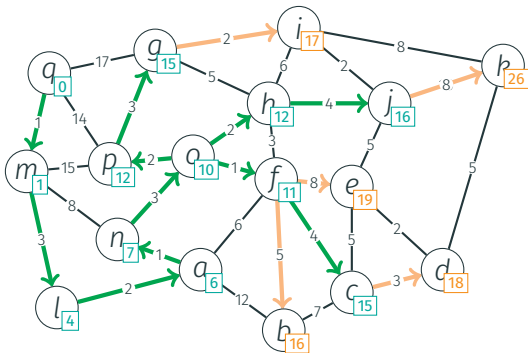


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[b,i,d,e,k]$. On étudie les voisins de j .

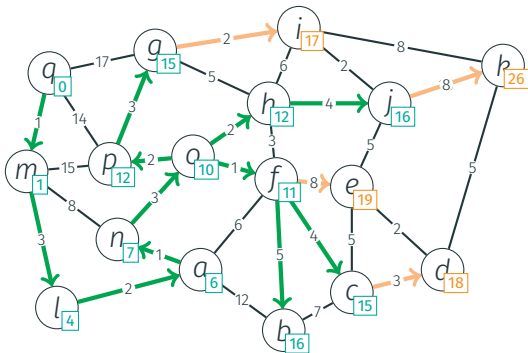


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[i,d,e,k]$. On étudie les voisins de b .

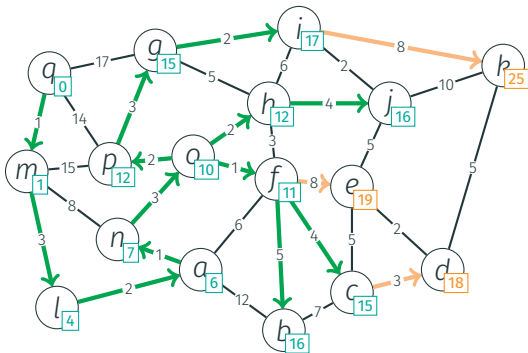


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[d,e,k]$. On étudie les voisins de i .

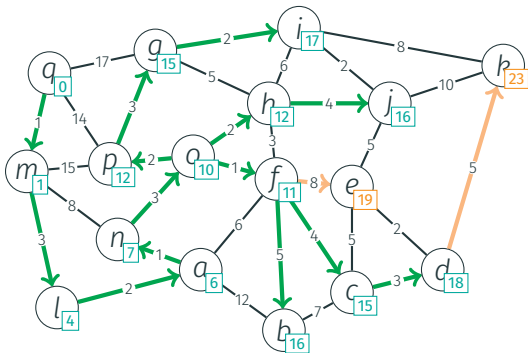


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[e,k]$. On étudie les voisins de d .

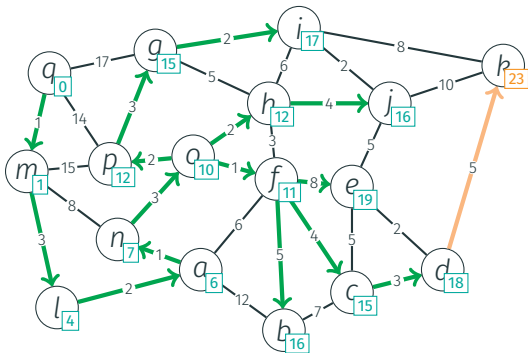


Algorithme de Dijkstra : illustration

Exemple

Plus courts chemins issus du sommet q .

État de la file à priorité : $[k]$. On étudie les voisins de e .

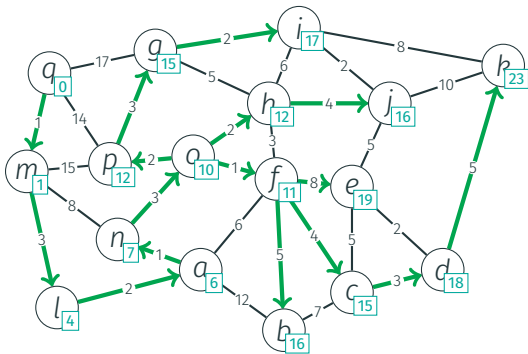


Algorithme de Dijkstra : illustration

Example

Plus courts chemins issus du sommet q .

État de la file à priorité : $\emptyset$. On étudie les voisins de k .



Théorème

L'algorithme de Dijkstra renvoie la longueur du plus court chemin entre s et tout sommet accessible depuis s .

Correction

Théorème

L'algorithme de Dijkstra renvoie la longueur du plus court chemin entre s et tout sommet accessible depuis s .

Démonstration.

Nous démontrons, en notant M l'ensemble des sommets marqués,

($\boxtimes_1$) père : $V \rightarrow V$ contient une **arborescence partielle** de chemins de racine s et **distance** $\pi(v)$ entre s et v .

($\boxtimes_2$) On a

$$\pi(v) = \min_{p \in M, \text{ préd de } v} \pi(p) + \rho(pv)$$

($\boxtimes_3$) On a $\pi(v) = d(s, v)$ pour $v \in M$.

Correction

Théorème

L'algorithme de Dijkstra renvoie la longueur du plus court chemin entre s et tout sommet accessible depuis s .

Démonstration.

Nous démontrons, en notant M l'ensemble des sommets marqués,

($\boxtimes_1$) père : $V \rightarrow V$ contient une **arborescence partielle** de chemins de racine s et **distance** $\pi(v)$ entre s et v .

($\boxtimes_2$) On a

$$\pi(v) = \min_{p \in M, \text{ préd de } v} \pi(p) + \rho(pv)$$

($\boxtimes_3$) On a $\pi(v) = d(s, v)$ pour $v \in M$.

($\boxtimes_1$) et ($\boxtimes_2$) se justifient par récurrence et inspection du code.



« On a $\pi(v) = d(s, v)$ pour $v \in M$. »

Preuve de (X_3)

« On a $\pi(v) = d(s, v)$ pour $v \in M$. »

À cause de (X_1), $\pi(v)$ représente la longueur d'un chemin, donc $\pi(v) \geq d(s, v)$. Supposons que pour un certain sommet v , nous marquons v avec $\pi(v) > d(s, v)$.

Preuve de (I_3)

« On a $\pi(v) = d(s, v)$ pour $v \in M$. »

À cause de (I_1), $\pi(v)$ représente la longueur d'un chemin, donc $\pi(v) \geq d(s, v)$. Supposons que pour un certain sommet v , nous marquons v avec $\pi(v) > d(s, v)$.

Soit C un chemin de s vers v de poids $d(s, v)$. Le prédécesseur u de v sur C ne peut pas être marqué. En effet, à cause de l'invariant (I_3), on aurait $\pi(u) = d(s, u)$, et à cause de l'invariant (I_2), on aurait $\pi(v) \leq \pi(u) + \rho(uv) = d(s, v)$.

Preuve de (I_3)

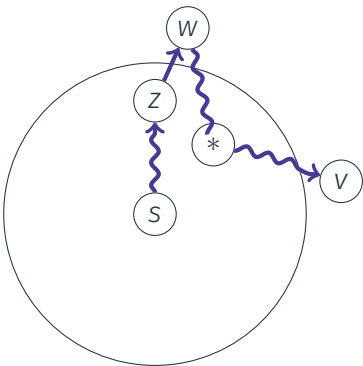
« On a $\pi(v) = d(s, v)$ pour $v \in M$. »

À cause de (I_1), $\pi(v)$ représente la longueur d'un chemin, donc $\pi(v) \geq d(s, v)$. Supposons que pour un certain sommet v , nous marquons v avec $\pi(v) > d(s, v)$.

Soit C un chemin de s vers v de poids $d(s, v)$. Le prédécesseur u de v sur C ne peut pas être marqué. En effet, à cause de l'invariant (I_3), on aurait $\pi(u) = d(s, u)$, et à cause de l'invariant (I_2), on aurait $\pi(v) \leq \pi(u) + \rho(uv) = d(s, v)$. Considérons le premier sommet w sur le chemin C tel que w est non marqué. Appelons z son prédécesseur, qui lui est marqué, ce qui implique notamment que

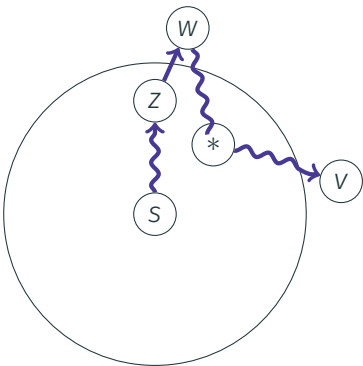
$$\pi(w) \leq \pi(z) + \rho(zw).$$

Preuve de (X_3) (suite)



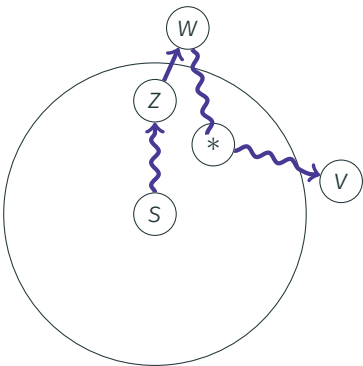
$$\pi(v) > d(s, v)$$

Preuve de (∇_3) (suite)



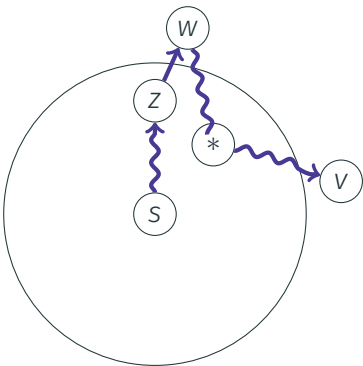
$$\begin{aligned}\pi(v) &> d(s, v) \\ &\geq d(s, w) \quad (\text{car } \rho \geq 0.)\end{aligned}$$

Preuve de ($\times_3$) (suite)



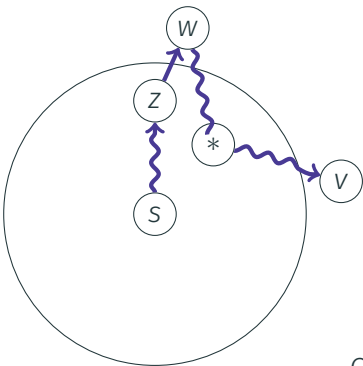
$$\begin{aligned}\pi(v) &> d(s, v) \\ &\geq d(s, w) \quad (\text{car } \rho \geq 0.) \\ &= d(s, z) + \rho(zw) \\ &\quad (\text{car } C \text{ plus court chemin.})\end{aligned}$$

Preuve de ($\times_3$) (suite)



$$\begin{aligned}\pi(v) &> d(s, v) \\ &\geq d(s, w) \quad (\text{car } \rho \geq 0.) \\ &= d(s, z) + \rho(zw) \\ &\quad (\text{car } C \text{ plus court chemin.}) \\ &= \pi(z) + \rho(zw) \quad (\text{car } z \in M.)\end{aligned}$$

Preuve de ($\times_3$) (suite)



$$\begin{aligned}\pi(v) &> d(s, v) \\ &\geq d(s, w) \quad (\text{car } \rho \geq 0.) \\ &= d(s, z) + \rho(zw) \\ &\quad (\text{car } C \text{ plus court chemin.}) \\ &= \pi(z) + \rho(zw) \quad (\text{car } z \in M.) \\ &\geq \pi(w) \quad (\text{comme vu plus haut.})\end{aligned}$$

Contredit le fonctionnement de la pile à priorité, car v a été extrait avant w !

Dans tous les cas, l'algorithme étudie les voisins de tout sommets, ce qui demande une complexité $O(m)$.

Complexité

Dans tous les cas, l'algorithme étudie les voisins de tout sommets, ce qui demande une complexité $O(m)$. Selon la manière dont la file est gérée, l'extraction du sommet de priorité minimal est $O(n)$ (liste) ou jusqu'à $O(\log n)$ (tas).

Dans tous les cas, l'algorithme étudie les voisins de tout sommets, ce qui demande une complexité $O(m)$. Selon la manière dont la file est gérée, l'extraction du sommet de priorité minimal est $O(n)$ (liste) ou jusqu'à $O(\log n)$ (tas). Ainsi

Proposition

La complexité de l'algorithme est

$$O(n^2) \quad \text{ou} \quad O(m + n \log n).$$

Complexité

Dans tous les cas, l'algorithme étudie les voisins de tout sommets, ce qui demande une complexité $O(m)$. Selon la manière dont la file est gérée, l'extraction du sommet de priorité minimal est $O(n)$ (liste) ou jusqu'à $O(\log n)$ (tas). Ainsi

Proposition

La complexité de l'algorithme est

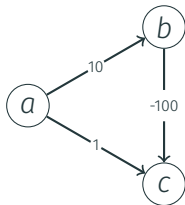
$$O(n^2) \quad \text{ou} \quad O(m + n \log n).$$

Les meilleures performances connues s'obtiennent avec un **tas de Fibonacci** (sujet non couvert dans ce cours) qui permet de décroître une priorité en coût amorti constant.

Nécessité de poids positifs

Contre-exemple

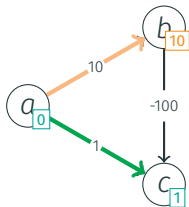
Dans le graphe



Nécessité de poids positifs

Contre-exemple

Dans le graphe

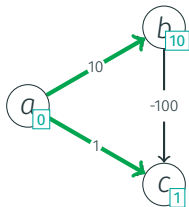


l'algorithme de Dijkstra au départ de *a*

Nécessité de poids positifs

Contre-exemple

Dans le graphe



l'algorithme de Dijkstra au départ de a se trompe. La distance entre a et c vaut -90 et non pas 1 .

Le cas général

L'algorithme de Bellman-Ford

1956



Lester Randolph Ford Jr
(1927+2017)

Principe

Initialiser les sommets à distance ∞ .

Répéter le calcul de $(*)$:

$$\text{dist}(s, v_i) = \min_{p \text{ préd de } v_i} \text{dist}(s, p) + \rho(p, v_i).$$

jusqu'à ce que les valeurs se stabilisent.

Algorithme de Bellman-Ford : pseudo-code

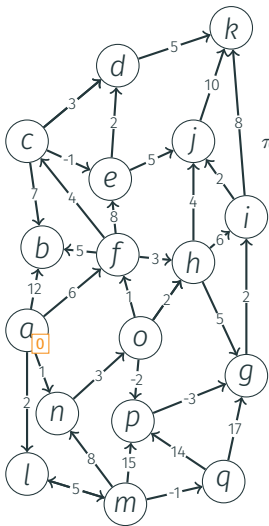
Algorithme 5 : Algorithme de Bellman-Ford

Entrées : Graphe pondéré $G = (V, E, \rho)$, sommet s

Résultat : Arborecence de plus courts chemins issus de s

```
1  $k \leftarrow 0; \pi^{(0)}(s) \leftarrow 0$ 
2 pour  $v \in V \setminus \{s\}$  faire
3    $\pi^{(0)}(v) \leftarrow +\infty$ 
4 répéter
5    $\text{changement} \leftarrow \text{faux}$ 
6    $k \leftarrow k + 1$ 
7   pour  $v \in V$  faire
8      $\pi^{(k)}(v) \leftarrow$ 
9        $\min(\pi^{(k-1)}(v), \min_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv))$ 
10    si  $\pi^{(k)}(v) \neq \pi^{(k-1)}(v)$  alors
11       $\text{changement} \leftarrow \text{vrai}$ 
12       $\text{père}(v) \leftarrow \operatorname{argmin}_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv)$ 
13 jusqu'à  $k = n$  ou  $\text{changement} = \text{faux}$ ;
14 si  $\text{changement} = \text{vrai}$  alors
15    $\text{il existe un circuit absorbant accessible depuis } s$ 
```

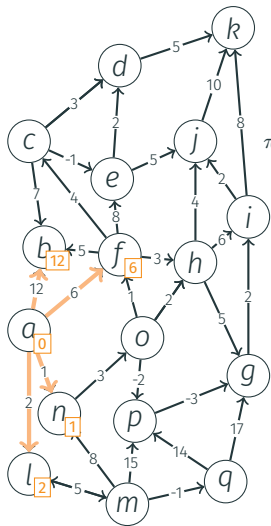
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv))$$

[illegible]

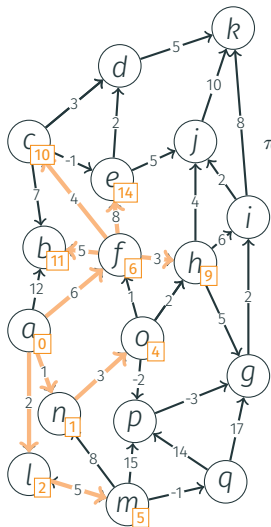
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞

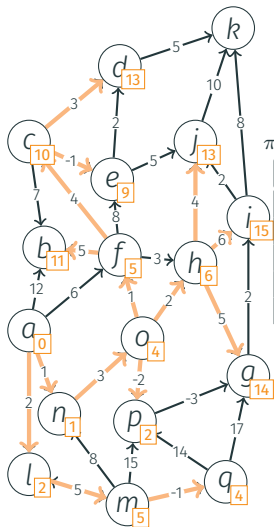
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞

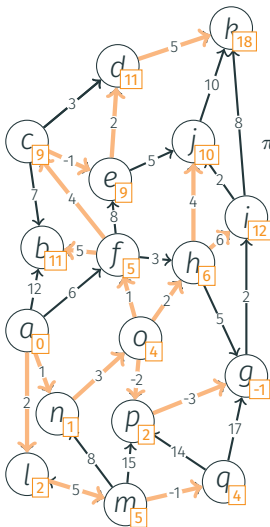
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ pr  d de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞
$\pi^{(3)}$	0	11	10	13	9	5	14	6	15	13	∞	2	5	1	4	2	4

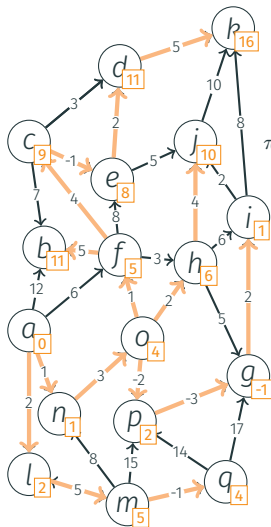
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ préd de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞
$\pi^{(3)}$	0	11	10	13	9	5	14	6	15	13	∞	2	5	1	4	2	4
$\pi^{(4)}$	0	10	9	11	9	5	-1	6	15	13	18	2	5	1	4	2	4

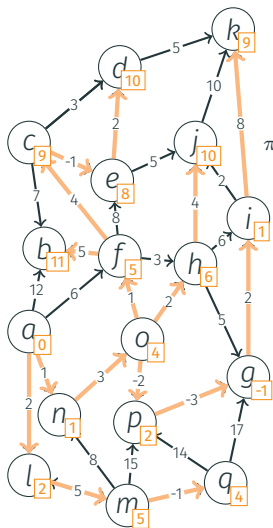
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ pr  d de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞
$\pi^{(3)}$	0	11	10	13	9	5	14	6	15	13	∞	2	5	1	4	2	4
$\pi^{(4)}$	0	10	9	11	9	5	-1	6	15	13	18	2	5	1	4	2	4
$\pi^{(5)}$	0	10	9	11	8	5	-1	6	1	13	16	2	5	1	4	2	4

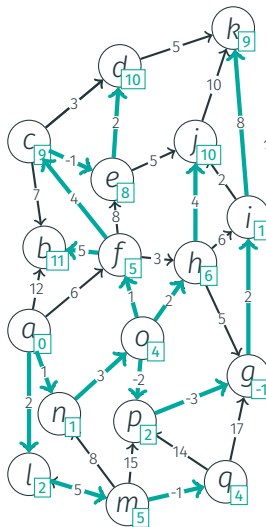
Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ pr  d de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞
$\pi^{(3)}$	0	11	10	13	9	5	14	6	15	13	∞	2	5	1	4	2	4
$\pi^{(4)}$	0	10	9	11	9	5	-1	6	15	13	18	2	5	1	4	2	4
$\pi^{(5)}$	0	10	9	11	8	5	-1	6	1	13	16	2	5	1	4	2	4
$\pi^{(6)}$	0	10	9	10	8	5	-1	6	1	13	9	2	5	1	4	2	4

Bellman-Ford : Illustration



$$\pi^{(k)}(v) \leftarrow \min(\pi^{(k-1)}(v), \min_{p \text{ pr  d de } v} \pi^{(k-1)}(p) + \rho(pv))$$

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q
$\pi^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\pi^{(1)}$	0	12	∞	∞	∞	6	∞	∞	∞	∞	∞	2	∞	1	∞	∞	∞
$\pi^{(2)}$	0	11	10	∞	14	6	∞	9	∞	∞	∞	2	5	1	4	∞	∞
$\pi^{(3)}$	0	11	10	13	9	5	14	6	15	13	∞	2	5	1	4	2	4
$\pi^{(4)}$	0	10	9	11	9	5	-1	6	15	13	18	2	5	1	4	2	4
$\pi^{(5)}$	0	10	9	11	8	5	-1	6	1	13	16	2	5	1	4	2	4
$\pi^{(6)}$	0	10	9	10	8	5	-1	6	1	13	9	2	5	1	4	2	4
$\pi^{(7)}$	0	10	9	10	8	5	-1	6	1	13	9	2	5	1	4	2	4

Il y a $O(n)$ étapes qui coûtent $O(m)$ opérations.

Il y a $O(n)$ étapes qui coûtent $O(m)$ opérations.

Proposition

La complexité est $O(nm)$.

Problème sources et destinations multiples

L'algorithme de Floyd-Warshall : principe

Principe

Numéroter les sommets $v_1, \dots, v_n$ dans un ordre quelconque.

Déterminer $\pi(v_i, v_j, r)$ le poids d'un plus court chemin $v_i \rightsquigarrow v_j$ qui n'utilise que $v_1, \dots, v_r$ comme sommets intermédiaires.

$$\pi(v_i, v_j, 0) = \begin{cases} 0 & \text{si } i = j \\ \rho(v_i v_j) & \text{si } v_i v_j \in E \\ +\infty & \text{sinon} \end{cases}$$
$$\pi(v_i, v_j, r) = \min \begin{pmatrix} \pi(v_i, v_r, r-1) + \pi(v_r, v_j, r-1) \\ \pi(v_i, v_j, r-1) \end{pmatrix}.$$

1962

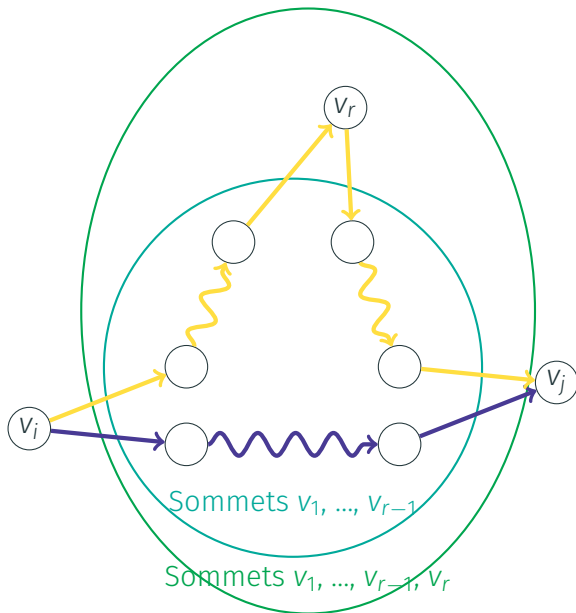


Robert W Floyd
(1936†2001)



Stephen Warshall
(1935†2006)

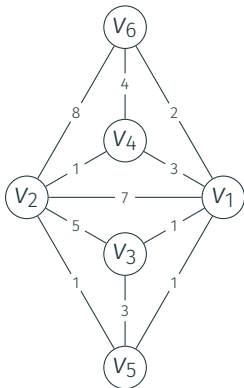
Algorithme de Floyd-Warshall : principe illustré



Implémentation de Floyd Warshall en python

```
class WeightedGraph():
    def __init__(self,n):
        self.n = n
        self.adjacency = [[sys.maxsize//2 for _ in range(n)]
                           ↪ for _ in range(n)]
    def add_edge(self, u, v, rho):
        self.adjacency[u][v] = rho
    def floydwarshall(self):
        D = deepcopy(self.adjacency)
        for u in range(self.n):
            D[u][u] = 0
        for r in range(self.n):
            for u in range(self.n):
                for v in range(self.n):
                    D[u][v] = min(D[u][v], D[u][r] + D[r][v])
        return(D)
```

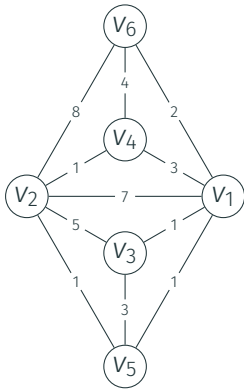
Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Illustration



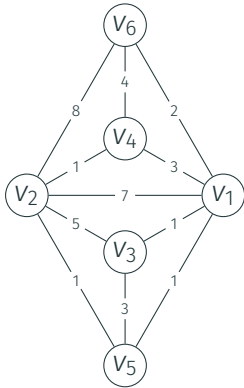
Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

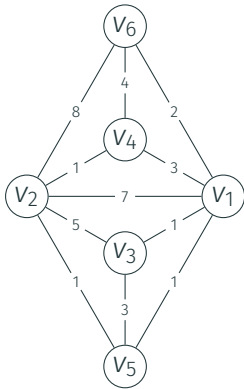
Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

Pour $r = 2$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

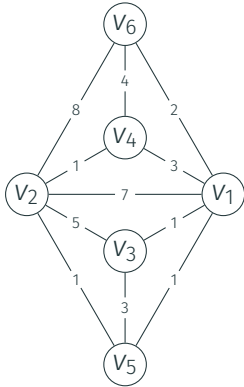
Pour $r = 2$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Pour $r = 3$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	6	1	3	1	2
v_2	6	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

Pour $r = 2$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

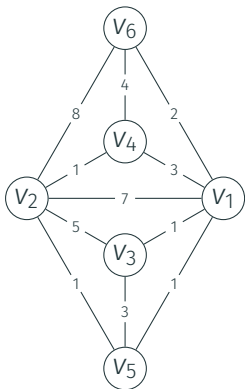
Pour $r = 3$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	6	1	3	1	2
v_2	6	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Pour $r = 4$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	4	1	3	1	2
v_2	4	0	5	1	1	5
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	5	3	4	3	0

Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

Pour $r = 2$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Pour $r = 3$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	6	1	3	1	2
v_2	6	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

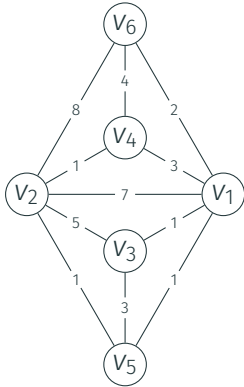
Pour $r = 4$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	4	1	3	1	2
v_2	4	0	5	1	1	5
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	5	3	4	3	0

Pour $r = 5$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	2	1	3	1	2
v_2	2	0	3	1	1	4
v_3	1	3	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	4	3	4	3	0

Illustration



Pour $r = 0$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	∞	3	∞
v_4	3	1	∞	0	∞	4
v_5	1	1	3	∞	0	∞
v_6	2	8	∞	4	∞	0

Pour $r = 1$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	4	4
v_5	1	1	2	4	0	3
v_6	2	8	3	4	3	0

Pour $r = 2$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	7	1	3	1	2
v_2	7	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Pour $r = 3$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	6	1	3	1	2
v_2	6	0	5	1	1	8
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	8	3	4	3	0

Pour $r = 4$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	4	1	3	1	2
v_2	4	0	5	1	1	5
v_3	1	5	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	5	3	4	3	0

Pour $r = 5$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	2	1	3	1	2
v_2	2	0	3	1	1	4
v_3	1	3	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	4	3	4	3	0

Pour $r = 6$:

	v_1	v_2	v_3	v_4	v_5	v_6
v_1	0	2	1	3	1	2
v_2	2	0	3	1	1	4
v_3	1	3	0	4	2	3
v_4	3	1	4	0	2	4
v_5	1	1	2	2	0	3
v_6	2	4	3	4	3	0

La complexité de l'algorithme de Floyd-Warshall est $O(n^3)$.

La complexité de l'algorithme de Floyd-Warshall est $O(n^3)$.

Il est plus intéressant d'utiliser un algorithme pour source unique, tel que l'algorithme de Dijkstra, si cela suffit.

Variante pour déterminer la relation d'accessibilité

Principe

Numéroter les sommets $v_1, \dots, v_n$ dans un ordre quelconque.

Déterminer la valeur de vérité $\gamma(v_i, v_j, r) \in \mathbb{B}$ de « il existe un chemin $v_i \rightsquigarrow v_j$ qui n'utilise que $v_1, \dots, v_r$ comme sommets intermédiaires ».

$$\gamma(v_i, v_j, 0) = \begin{cases} V & \text{si } i = j \\ V & \text{si } v_i v_j \in E \\ F & \text{sinon} \end{cases}$$

$$\gamma(v_i, v_j, r) = \gamma(v_i, v_r, r-1) \wedge \gamma(v_r, v_j, r-1) \vee \gamma(v_i, v_j, r-1)$$